Lecture 4 — UKF vs Linear KF

Exercise 1: Scalar Nonlinear System

We are given a scalar system, nonlinear in **both** transition f and measurement h. A linear approximation is obtained by setting $\varphi_f = \varphi_h = 0$ and $\sin(x) \approx x$:

Nonlinear	**Linear approximation**
$x_k = a\sin(x_{k-1} + \varphi_f) + bu_{k-1} + w_{k-1}$	$x_k = ax_{k-1} + bu_{k-1} + w_{k-1}$
$z_k = \sin(cx_k + \varphi_h) + v_k$	$z_k = cx_k + v_k$

For both

$w \in \text{NID}(0, Q)$ Process noise, $v \in \text{NID}(0, R)$ Measurement noise, $x_0 \in \mathcal{N}(\hat{x}_0,\, P_0)$Initial guess

The standard linear KF is applied to the linear system, while the UKF is applied directly to the nonlinear system — no linearisation needed.

Why the linear KF cannot handle nonlinear systems

The linear KF operates entirely through matrix multiplication:

$$x_{k+1} = \mathbf{\Phi} x_k + Lu_k, \qquad z_k = Hx_k$$

which can only scale and rotate — never bend or curve.

The deeper problem is **covariance propagation**: the KF assumes uncertainty remains Gaussian after passing through f and h, but a nonlinear function distorts a Gaussian into a non-Gaussian shape — the mean and covariance alone no longer fully describe it. Forcing a nonlinear f or h into $\mathbf{\Phi}$ and H introduces linearisation error that grows as the state moves away from the operating point, and the propagated covariance increasingly misrepresents the true uncertainty shape.

UKF — handling nonlinearity without linearisation

The state estimate (mean) can still be pushed directly through f and h, but the covariance no longer describes the correct shape of the uncertainty after a nonlinear transformation — a Gaussian mapped through a nonlinearity becomes skewed or multimodal. **Meaning the mean and covariance alone are insufficient to fully characterise it.**

Both **EKF** and **UKF** approximate a solution by still assuming Gaussian uncertainty, but differ in how accurately they propagate the covariance through the nonlinearity.

We use the UKF here for two reasons:

- **Higher accuracy:** the EKF handles nonlinearity by linearising f and h around the current estimate using the **Jacobian**:

  $$F = \left.\frac{\partial f}{\partial x}\right|_{\hat{x}_k}, \qquad H = \left.\frac{\partial h}{\partial x}\right|_{\hat{x}_k}$$

  This is a 1st-order approximation — it captures the local slope but ignores curvature (2nd-order effects), so the propagated covariance is only approximately correct. The UKF instead propagates sigma points directly through f and h, reconstructing the covariance from the spread of the transformed points — capturing up to 2nd-order effects without any derivatives.

- **No Jacobian required:** computing F and H analytically is tedious and error-prone for complex functions. The UKF treats f and h as black-box functions — only evaluations are needed, not derivatives.

Example of this skewed thing

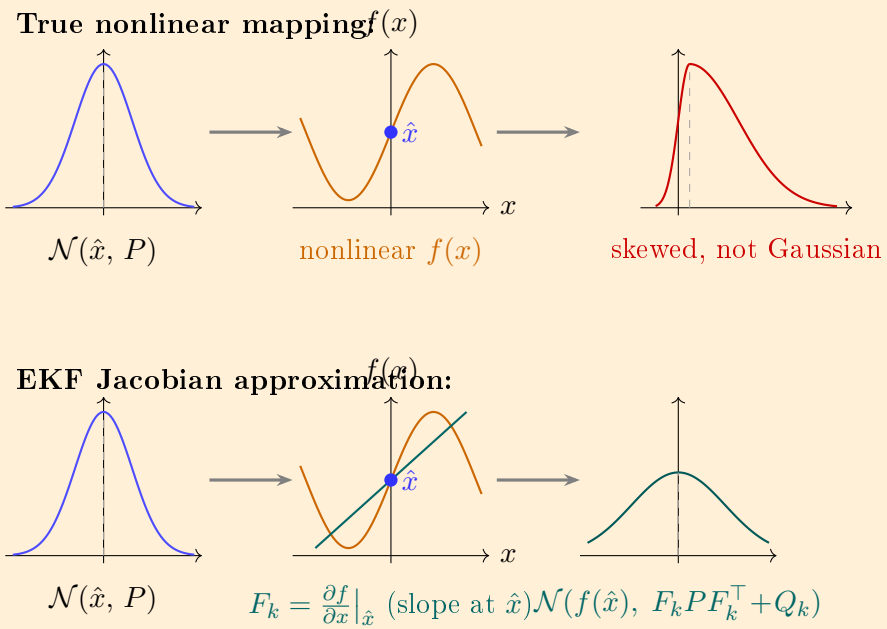

Figure 1: Top row: the true nonlinear f warps the Gaussian into a skewed non-Gaussian — no simple covariance formula exists. Bottom row: the EKF draws a tangent line on f at $\hat{x}$, giving slope F_k (the Jacobian). The uncertainty P is then propagated through that slope as $F_k P F_k^\top + Q_k$, keeping the output Gaussian — accurate near $\hat{x}$, increasingly wrong further away.

UKF - Step by Step

The UKF can be seen as a smarter Monte Carlo approach — instead of randomly sampling many points, we place $2n+1$ sigma points at carefully chosen locations that capture the current state estimation uncertainty P (how uncertain we are about where the true state actually is, given all measurements seen so far).

Each sigma point is passed through the nonlinear f and h directly, and weighted contributions from all points reconstruct both the predicted mean (state estimate) and the predicted covariance. The weights control how much each point influences both — points closer to the mean carrying more weight than those at the edges.

Step 1 Parameters: default values ()$\alpha = 1$, $\kappa = 2$, $\beta = 0$ give $\lambda = 2$)

$$\lambda = \alpha^2(n+\kappa) - n, \qquad k = \sqrt{n+\lambda}, \qquad n = \dim(x)$$

These points can be disrbuted differnt depending on set parametars

- λ is a scaling parameter controlling how far the sigma points are spread from the mean.
- k is the actual distance multiplier used when placing them.

> With the default values and $n = 2$ states: $\lambda = 1^2(2+2) - 2 = 2$, $k = \sqrt{2+2} = 2$.

The larger n is, the further out the points are placed to cover the wider spread of a higher-dimensional Gaussian.

Step 2 — Sigma points (eigendecomposition)

The scaled covariance $(n+\lambda)P$ is decomposed via `np.linalg.eigh`:

$$(n+\lambda)P = VLV^\top, \qquad S = V\sqrt{L}$$

where:

- $L = \text{diag}(l_1, \ldots, l_n)$ — eigenvalues, variance magnitude in each direction
- $V = [u_1, \ldots, u_n]$ — eigenvectors, which directions the uncertainty points in
- $S = V\sqrt{\Lambda}$ — matrix square root of $(n+\lambda)P$, each column $S_{:,i} = u_i\sqrt{l_i}$ gives the offset direction and magnitude

The $2n+1$ sigma points are placed symmetrically around the mean (also written as $\mu_x = \hat{x}$, $k = \sqrt{n+\lambda}$ in lecture notation):

$$SigmaPT_i = \begin{cases} \mu_x \quad = \hat{x} & i = 0 \quad \text{(center: current mean estimate)} \\ \mu_x + u_i\sqrt{l_i} \quad = \hat{x} + S_{:,i} & 1 \le i \le n \quad \text{(pushed positive along } u_i\text{)} \\ \mu_x - u_{i-n}\sqrt{l_{i-n}} \quad = \hat{x} - S_{:,i-n} & n+1 \le i \le 2n \quad \text{(symmetric negative push)} \end{cases}$$

> The `np.maximum(eigvals, 0)` clips any floating-point negative eigenvalues before the square root — P should be positive definite but numerical drift can produce tiny negatives after many steps. Which could break the filter — the clip prevents this.

step 3: Weighting of Mean and Covariance

Each sigma point is assigned two weights — one controlling its contribution to the predicted mean, one to the predicted covariance:

$$\underbrace{w_i^m = \begin{cases} \dfrac{\lambda}{n+\lambda} & i = 0 \\ \dfrac{1}{2(n+\lambda)} & 1 \le i \le 2n \end{cases}}_{\text{weight mean}} \qquad \underbrace{w_i^c = \begin{cases} \dfrac{\lambda}{n+\lambda} + (1-\alpha^2+\beta) & i = 0 \\ \dfrac{1}{2(n+\lambda)} & 1 \le i \le 2n \end{cases}}_{\text{weight covariance}}$$

The centre point $\sigma_0 = \hat{x}$ gets a higher weight since it sits exactly at the mean estimate. The covariance weight W_0^c has the extra term $(1-\alpha^2+\beta)$ to allow fine-tuning of how much the centre point contributes to the spread — $\beta = 2$ is optimal for Gaussian distributions. All weights sum to 1.

> **Weights are fixed** — computed once from α, κ, β, n and reused every step. Two sets exist so the centre point's contribution to the covariance can be tuned independently via $(1 - \alpha^2 + \beta)$, without affecting the mean weight.
>
> **Example** $(n = 1, \alpha = 1, \lambda = 1, \beta = 2)$:
>
> - $\underbrace{W_0^m}_{\text{centre}} = \frac{1}{2} = 0.5, \qquad \underbrace{W_{1,2}^m}_{\text{outer points}} = \frac{1}{4} = 0.25 \quad \rightarrow \text{sum} = 1$ ✓
> - $\underbrace{W_0^c}_{\text{centre}} = 0.5 + \underbrace{(1 - 1 + 2)}_{\text{tuning term}} = 2.5, \qquad \underbrace{W_{1,2}^c}_{\text{outer points}} = 0.25 \quad \rightarrow$ centre weighted more

Step 4 - Propagate and compute output statistics.

Each of the $2n + 1$ sigma points is pushed through the nonlinear f together with the known input u — no Jacobian, no approximation:

$$\sigma_i^{\text{pred}} = f(\sigma_i,\, u), \qquad i = 0, 1, \ldots, 2n$$

where u is identical for all points since it is a known deterministic input — only the state uncertainty σ_i varies across the points.

We then use the weights to reconstruct the predicted mean and covariance from the spread of the propagated points:

Predicted state mean

weighted average of where the sigma points land after f:

$$\hat{x}_{k+1|k} = \sum_{i=0}^{2n} W_i^m \, \sigma_i^{\text{pred}}$$

Predicted state covariance

weighted sum of how far each propagated sigma point is from the predicted mean, giving the spread of uncertainty after f, plus the irreducible process noise Q:

$$P_{k+1|k} = Q + \sum_{i=0}^{2n} W_i^c \underbrace{\left(\sigma_i^{\text{pred}} - \hat{x}_{k+1|k}\right)\left(\sigma_i^{\text{pred}} - \hat{x}_{k+1|k}\right)^\top}_{\text{outer product: spread of point } i \text{ from mean}}$$

Because Q adds irreducible process noise each step, and the nonlinear f warps the sigma points unevenly — the curve is steeper in some places and flatter in others, so the outer points land at different distances from the predicted mean $\hat{x}_{k+1|k}$, breaking the symmetry and generally increasing the spread compared to a linear propagation.

Still Step 4 Just Now on the measurement side

From the system equation $z_k = h(x_k) + v_k$, it may seem natural to simply push the predicted state $\hat{x}_{k+1|k}$ through h. However $\hat{x}_{k+1|k}$ is only the **single weighted mean** — one point that discards all uncertainty information.

Instead we push all $2n+1$ propagated sigma points through h:

$$z_{\text{sig},i} = h(\sigma_i^{\text{pred}}), \qquad i = 0, \ldots, 2n$$

This preserves the full spread of uncertainty through the nonlinear h, following the same logic as Step 4.

> the sigma points represent the distribution of x_k, so passing them through h gives the distribution of z_k. Collapsing to $\hat{x}_{k+1|k}$ first and then pushing through h would lose all covariance information and reduce to a linear-style update.

We then use actually these $z_{\text{sig},i}$ to calcualte the weighted measurements.

predicted measurement mean

weighted average of where the sigma points land after h using same wiegth mean as from the **predicted state mean**:

$$\hat{z}_{k+1|k} = \sum_{i=0}^{2n} W_i^m \, z_{\text{sig},i}$$

S_{zz} **increases** because we add R on top of the sigma point spread — the sensor always adds irreducible noise regardless of how well the sigma points capture the state uncertainty.

Innovation covariance

Total measurement uncertainty, analogous to $S = HPH^\top + R$ in the linear KF but computed from the spread of sigma points through h rather than through H:

$$S_{zz} = R + \sum_{i=0}^{2n} W_i^c \left(z_{\text{sig},i} - \hat{z}_{k+1|k}\right)\left(z_{\text{sig},i} - \hat{z}_{k+1|k}\right)^\top$$

Cross-covariance

replaces $PH^\top$ from the linear KF:

$$P_{xz} = \sum_{i=0}^{2n} W_i^c \underbrace{\left(\sigma_i^{\text{pred}} - \hat{x}_{k+1|k}\right)}_{\text{state deviation}} \underbrace{\left(z_{\text{sig},i} - \hat{z}_{k+1|k}\right)^\top}_{\text{measurement deviation}}$$

It answers: **if the measurement is off $\left(z_{\text{sig},i} - \hat{z}_{k+1|k}\right)^\top$, how much should the state be corrected?**

A large P_{xz} means state and measurement move together — so a measurement error reliably signals a state error.

A small P_{xz} means they are weakly linked — the measurement tells you little about the state.

Step 5 — Measurement update (identical to linear KF)

Once P_{xz} and S_{zz} are formed, the update is standard:

$$K_k = P_{xz}\, S_{zz}^{-1} \quad \text{Model uncertianty/total uncertianty}$$

$$\hat{x}_{k|k} = \hat{x}_{k+1|k} + K_k\,(z_k - \hat{z}_{k+1|k}) \quad \text{estimate of this state based of k data}$$

$$P_{k|k} = P_{k+1|k} - K_k\, S_{zz}\, K_k^\top \quad \text{Covariance uncertianty}$$

The UKF and linear KF are identical from this point — the only difference is how P_{xz} and S_{zz} were computed: through sigma points vs. through H and Φ directly.

> **Uncertainty decreases** because seeing a measurement tells us something about the state — we subtract $K_k S_{zz} K_k^\top$ which is always positive, so $P_{k|k} < P_{k+1|k}$. The more the measurement is linked to the state (large P_{xz}), the more uncertainty is removed. The UKF and linear KF are identical from this point — the only difference is how P_{xz} and S_{zz} were computed: sigma points vs. H and Φ.

Results — Exercise parameters

Initial parameters:

$$a = 0.95, \quad k = 1, \quad b = k(1-a), \quad c = 1, \quad \varphi_f = \varphi_h = 0, \quad f_u = 0.02\,\text{Hz}$$

Since $u = \sin(2\pi f_u t) \in [-1, 1]$ and the state settles around $x^* \approx k \cdot u$, the state stays roughly in $[-1, 1]$. We can therefore check how well $\sin(cx + \varphi) \approx cx + \varphi$ holds by evaluating at $x = 1$ for each case:

Case	**True** $h(x)$	**Linear approx.**	**Error**
(a) $c = 1,\ \varphi = 0$	$\sin(1) = 0.841$	1	$0.159 \approx 16\%$
(b) $\varphi_h = \frac{\pi}{16}$	$\sin(1.196) = 0.932$	1.196	$0.264 \approx 22\%$
(c) $\varphi_f = \frac{\pi}{16}$	$\sin(1.196) = 0.932$	1.196	$0.264 \approx 22\%$
(d) $c = 10$	$\sin(10) = -0.544$	10	$10.544 \approx 105\%$

(a) $c = 1$**,** $\varphi_f = \varphi_h = 0$**:** 16% error but the filters still perform similarly — the approximation is poor but consistent, so the linear KF does not diverge, just accumulates small errors.

(b) $\varphi_h = \frac{\pi}{16}$**:** shifts the measurement operating point, worsening the approximation to 22%. The linear KF assumed $\varphi_h = 0$ in $H = c$, so it systematically misreads the measurement. The UKF handles the offset naturally.

(c) $\varphi_f = \frac{\pi}{16}$**:** same error magnitude as (b) but now in the transition function — the linear KF accumulates prediction error each step since $\mathbf{\Phi} = a$ assumed no phase offset in f.

(d) $c = 10$**:** by far the largest difference — 105% error means the linear approximation is completely invalid. The linear KF predicts $h(x) \approx 10$ while the true output is $\sin(10) = -0.544$. The UKF still tracks well since sigma points are pushed through the true h directly.

> Case (d) is where the UKF most clearly outperforms the linear KF — the argument $10x$ grows far outside the region where $\sin(10x) \approx 10x$ holds, and no amount of tuning can fix a 105% linearisation error.